

C Programming Practice Set for Digital Design / ASIC Interviews

VOLUME 2

All-new questions on the same topics as Volume 1 — Sections 1, 2, 3, 4, 5, 6, 8 and 9.
Nothing here repeats Volume 1; every question is fresh so you can test whether the *technique* transferred, not whether you memorised an answer.

Section map (matches Volume 1 numbering)

- 1 — Bitwise Operations & Bit Manipulation
- 2 — Data Representation, sizeof, Endianness & Padding
- 3 — Pointers, volatile & Memory-Mapped Registers
- 4 — Output-Tracing Traps
- 5 — Hardware-Flavored Coding Problems
- 6 — Structs, Unions & Register Maps
- 8 — More Bit Tricks & Digital-Design Idioms
- 9 — More Output-Tracing & Memory Traps

Rule for Volume 2: attempt every question on paper with a pen, in binary or hex where relevant, before reading a single line of the solution. If you can derive it rather than recall it, you are ready.

Section 1 — Bitwise Operations & Bit Manipulation

Same conventions as Volume 1: bit 0 is the LSB, 32-bit unsigned `int` unless stated.

Q1. Write a macro/function that extracts the bit range [hi:lo] of a 32-bit word and returns it right-justified. Handle the full-width case safely.

Solution

```
/* Naive - breaks when hi-lo+1 == 32 because 1u<<32 is undefined */
#define GET_FIELD_BAD(x,hi,lo) (((x) >> (lo)) & ((1u << ((hi)-(lo)+1)) - 1))

/* Safe: build the mask by shifting all-ones DOWN instead */
static inline unsigned get_field(unsigned x, int hi, int lo)
{
    unsigned width = (unsigned)(hi - lo + 1);
    unsigned mask = 0xFFFFFFFFu >> (32 - width); /* width in 1..32 */
    return (x >> lo) & mask;
}
```

Shifting by an amount $\geq$ the type width is undefined behaviour, so a 32-bit-wide field breaks the textbook mask. Shifting all-ones right by $(32 - \text{width})$ never shifts by 32 when $\text{width} \geq 1$. Interviewers love asking 'what if the field is the whole register?' after you give the naive answer.

Q2. Given a mask m, merge bits from b where m is 1 and keep bits from a where m is 0. Give the two-operation form and explain why it is a mux.

Solution

```
r = (a & ~m) | (b & m); /* obvious form: 4 operations */
r = a ^ ((a ^ b) & m); /* classic form: 3 operations */
```

Check the XOR form bit by bit: where $m=1$ the result is $a \wedge (a \wedge b) = b$; where $m=0$ it is $a \wedge 0 = a$. That is precisely a 2:1 multiplexer with m as the select line, done 32 bits at a time — the software equivalent of `assign r = (a & ~m) | (b & m);` in Verilog.

Q3. Set a field of width w at position p to value v, in ONE expression, guarding against v being too large.

Solution

```
#define FIELD_MASK(w,p) (((1u << (w)) - 1u) << (p))
#define FIELD_SET(x,w,p,v) (((x) & ~FIELD_MASK(w,p)) | \
                             (((unsigned)(v) << (p)) & FIELD_MASK(w,p)))

/* e.g. write 3 into bits [5:4] of REG */
REG = FIELD_SET(REG, 2, 4, 3);
```

Three ideas graded here: clear before you OR (otherwise you can only ever set bits, never clear them); mask v after shifting so an out-of-range value cannot spill into neighbouring fields; parenthesise every macro argument. The out-of-range spill is a real silicon bug — writing 5 into a 2-bit field would corrupt bit 6.

Q4. Isolate the HIGHEST set bit of x (the opposite of $x \& \sim x$), and separately clear it.

Solution

```
unsigned highest_bit(unsigned x) /* returns e.g. 0b0100000 for 0b0101101 */
{
    x |= x >> 1; x |= x >> 2; x |= x >> 4;
    x |= x >> 8; x |= x >> 16; /* smear MSB downwards */
    return x - (x >> 1); /* keep only the top bit */
}

unsigned clear_highest(unsigned x) { return x & ~highest_bit(x); }
```

The OR-cascade turns `0b0101101` into `0b0111111`; subtracting its right-shift (`0b0011111`) leaves `0b0100000`. Isolating the lowest set bit is one operation ($x \& \sim x$) but the highest needs this smear — a useful

asymmetry to be able to explain. Used for finding the MSB index in priority encoders and normalisation.

Q5. Set or clear a bit depending on a flag, WITHOUT any if/else or ternary.

Solution

```
/* flag != 0 -> set bit n ; flag == 0 -> clear bit n */
x = (x & ~(1u << n)) | ((unsigned)(-(!flag)) & (1u << n));

/* equivalent, using the mux identity from Q2 */
unsigned target = (unsigned)(-(!flag)); /* 0x00000000 or 0xFFFFFFFF */
x = x ^ ((x ^ target) & (1u << n));
```

!!flag normalises any nonzero value to exactly 1; negating gives an all-zeros or all-ones mask. Branchless conditional updates matter in ISRs and tight polling loops where a mispredicted branch costs more than the arithmetic — and they map one-to-one onto combinational logic.

Q6. Implement a saturating 8-bit add: the result must clamp at 255 instead of wrapping. Give a branchless version.

Solution

```
unsigned char sat_add8(unsigned char a, unsigned char b)
{
    unsigned s = (unsigned)a + b; /* up to 510, no overflow in int */
    return (unsigned char)(s | (unsigned)(-(int)(s >> 8)));
}
```

If $s > 255$ then bit 8 is set, so $s \gg 8$ is 1 and $-(1)$ is all-ones: ORing forces every low bit to 1, and truncation to unsigned char gives 0xFF. If $s \leq 255$ the OR is with 0 and s passes through. Saturating arithmetic is standard in DSP datapaths — being asked to write it branchlessly is common for signal-processing-adjacent ASIC roles.

Q7. Count how many bits are set within the range [hi:lo] of a word.

Solution

```
int popcount_range(unsigned x, int hi, int lo)
{
    unsigned width = (unsigned)(hi - lo + 1);
    unsigned f = (x >> lo) & (0xFFFFFFFFu >> (32 - width));
    int c = 0;
    while (f) { f &= f - 1; c++; } /* Kernighan */
    return c;
}
```

Composition of two Volume 1 techniques: safe field extraction, then Kernighan popcount. Interviewers frequently build questions by composing two primitives — recognising the parts is the skill being tested. Practical use: counting active channels in a 32-bit interrupt-pending register belonging to one peripheral group.

Q8. Return 1 if two values differ in EXACTLY one bit. Why is this useful for Gray-code and one-hot checking?

Solution

```
int differ_by_one_bit(unsigned a, unsigned b)
{
    unsigned d = a ^ b;
    return d != 0 && (d & (d - 1)) == 0; /* d is a power of two */
}
```

XOR marks the differing positions; requiring the result to be a power of two means exactly one position differs. This is the property that DEFINES Gray code (consecutive codes differ in one bit) — so this function is exactly the assertion you would write in a testbench checking a Gray-coded FIFO pointer, or checking that a one-hot state vector advanced legally.

Q9. Multiply by 7, by 15 and by 9 using only shifts, adds and subtracts.

Solution

```
y = (x << 3) - x;          /* x*7 = 8x - x */
y = (x << 4) - x;          /* x*15 = 16x - x */
y = (x << 3) + x;          /* x*9 = 8x + x */
y = (x << 6) + (x << 3) + x; /* x*73 = 64x + 8x + x */
```

Constant multiplication decomposes into the binary representation of the constant: shifts for the set bits, plus subtraction when the constant is just below a power of two ($7 = 8 - 1$ saves an add). This is exactly what a synthesis tool does for a constant multiplier, and what compilers do with $x * 7$ — showing you know it signals hardware literacy.

Q10. TRICKY — What is the output? (Shift-count and type-width trap)

```
#include <stdio.h>
int main(void)
{
    unsigned char b = 0x80;
    unsigned int x = 1;
    printf("%X\n", b << 1);          /* line 1 */
    printf("%X\n", (unsigned char)(b << 1));
    printf("%X\n", x << 32);         /* line 3 */
    printf("%X\n", (unsigned)(-1) >> 1);
    return 0;
}
```

Solution

Line 1: **100** — b is promoted to `int` before shifting, so the 1 is NOT lost; the result is `0x100`, not 0. Line 2: **0** — the explicit cast back to 8 bits discards it. Line 3: **undefined behaviour** (shift count equals the width); typical x86 output is 1 because the CPU masks the count to 5 bits, but you must call it UB. Line 4: **7FFFFFFF** — logical shift of an unsigned. Integer promotion silently widening your 8-bit value is the trap: in hardware an 8-bit shift loses the bit, in C it does not.

Section 2 — Data Representation, sizeof, Endianness & Padding

Q11. Predict sizeof for each struct on a 64-bit machine and give the byte layout. Then reorder s3 to be as small as possible.

```
struct s1 { short a; char b; short c; };
struct s2 { char a; short b; int c; double d; };
struct s3 { char a; double b; short c; int d; char e; };
```

Solution

```
s1: a(2) b(1) pad(1) c(2) pad(2)          = 8
s2: a(1) pad(1) b(2) c(4) d(8)            = 16
s3: a(1) pad(7) b(8) c(2) pad(2) d(4) e(1) pad(7) = 32
s3 reordered { double b; int d; short c; char a, e; }:
    b(8) d(4) c(2) a(1) e(1)              = 16
```

Each member starts at an address that is a multiple of its own size; the total is rounded up to the largest member's alignment (8 here, because of the double). Sorting members largest-to-smallest halves `s3` from 32 to 16 bytes. In a packet or register-map struct this matters twice over: wasted RAM, and offsets that silently differ from the hardware spec.

Q12. What are the byte contents, in address order, of the 64-bit value 0x0102030405060708 on little-endian and big-endian machines? Write code that prints them.

Solution

```
#include <stdio.h>
#include <stdint.h>
```

```
int main(void)
{
    uint64_t v = 0x0102030405060708ULL;
    unsigned char *p = (unsigned char *)&v;
    for (int i = 0; i < 8; i++) printf("%02X ", p[i]);
    return 0;
}
```

Little-endian: **08 07 06 05 04 03 02 01**. Big-endian: **01 02 03 04 05 06 07 08** (the value written left to right, hence 'network byte order'). Note the extra trap in 64-bit land: some older ARM configurations used mixed/middle-endian for doubles, so for anything crossing a chip boundary you serialise byte by byte with explicit shifts rather than memcpy-ing a struct.

Q13. TRICKY — What is the output? (Integer promotion in an expression)

```
#include <stdio.h>
int main(void)
{
    unsigned char a = 200, b = 100;
    unsigned char c = a + b;
    int d = a + b;
    printf("%d %d\n", c, d);

    short s = -1;
    unsigned short us = 0xFFFF;
    printf("%d %d\n", s == us, (int)s == (int)us);
    return 0;
}
```

Solution

Line 1: **44 300**. Both operands promote to int, so the sum really is 300; storing it in an unsigned char truncates modulo 256 ($300 - 256 = 44$), while the int keeps it. Line 2: **0 0**. Both short types promote to int: s becomes -1, us becomes 65535, so they are never equal. The lesson for hardware code: C computes in int width no matter how narrow your variables are; if you want 8-bit wraparound you must store back into an 8-bit type (or mask), exactly as you would size a wire in RTL.

Q14. Two's complement corner cases: what are the values and which of these are undefined?

```
int a = INT_MIN;           /* -2147483648 */
int b = -a;                /* ?? */
int c = abs(a);            /* ?? */
unsigned u = (unsigned)a;  /* ?? */
int d = a / -1;            /* ?? */
```

Solution

b, c and d are all **undefined behaviour**: +2147483648 is not representable in a 32-bit int, so negating, abs-ing or dividing INT_MIN by -1 overflows (on x86, $a / -1$ actually raises a hardware exception). Only u is well defined: conversion to unsigned is modular, giving **2147483648** (0x80000000). Root cause: two's complement is asymmetric — one more negative value than positive. Hardware people should know this because saturating and absolute-value units must special-case exactly this code.

Q15. What does this print, and what does it tell you about how floats compare? (IEEE-754 awareness)

```
#include <stdio.h>
int main(void)
{
    float a = 0.1f, b = 0.2f;
    printf("%d\n", (a + b) == 0.3f);
    printf("%.10f\n", a + b);
    printf("%d\n", (int)(0.1f * 3 * 10));
    return 0;
}
```

Solution

Prints **0**, then about **0.3000000119**, then **2**. Neither 0.1 nor 0.2 is exactly representable in binary floating point, so the sum is slightly off and the equality fails; the last line rounds down to 2 because the product is 2.9999... and casting truncates. Correct practice: compare with a tolerance ($\text{fabsf}(x-y) < 1\text{e-}6\text{f}$), and in fixed-point hardware modelling prefer integer/fixed-point arithmetic entirely — which is why most RTL reference models in C avoid float.

Q16. What is the size and layout of this bit-field struct? Why can the answer differ between compilers?

```
struct flags {
    unsigned int  a : 3;
    unsigned int  b : 6;
    unsigned int   : 0;      /* special: force next to a new unit */
    unsigned int  c : 5;
};
```

Solution

On gcc/x86 this is **8 bytes**: a and b share the first 4-byte unit (3+6 = 9 bits used), the anonymous zero-width field forces c to start a new unit, so c occupies a second 4-byte unit. What is implementation-defined: whether allocation starts from the LSB or the MSB end, whether fields may straddle unit boundaries, and the padding rules. That is why register maps use explicit masks and shifts instead — the same struct can map field a to bits [2:0] on one toolchain and [31:29] on another.

Q17. TRICKY — What is the output? (char signedness and right shift)

```
#include <stdio.h>
int main(void)
{
    char c = 0x80;                      /* signed char on x86 gcc */
    unsigned char u = 0x80;
    printf("%d %d\n", c >> 4, u >> 4);
    printf("%X %X\n", (unsigned)c, (unsigned)u);
    return 0;
}
```

Solution

Line 1: **-8 8**. c is -128; promotion to int gives 0xFFFFF80, and the arithmetic right shift keeps the sign: 0xFFFFF8 = -8. u is 128, shifting gives 8. Line 2: **FFFFFF80 80**. Plain char has implementation-defined signedness (signed on x86, unsigned on many ARM toolchains), so byte-manipulation code MUST say unsigned char or uint8_t. This single bug — a sign-extended sensor byte — is one of the most common in embedded code.

Q18. Given a 32-bit float, extract its sign, exponent and mantissa fields without using a union (C++-safe method).

Solution

```
#include <string.h>
#include <stdint.h>
void split(float f, int *sign, int *exp_, uint32_t *mant)
{
    uint32_t u;
    memcpy(&u, &f, sizeof u);          /* legal in C and C++ */
    *sign = (int)(u >> 31);              /* bit 31 */
    *exp_ = (int)((u >> 23) & 0xFFu);    /* bits [30:23] */
    *mant = u & 0x7FFFFFFu;              /* bits [22:0] */
}
/* for 1.0f : sign=0, exp=127 (bias), mant=0 -> 0x3F800000 */
```

This is Volume 1's field-extraction idiom applied to IEEE-754: shift down, mask to width. Bias 127 means the stored exponent 127 represents 2^0 . Worth stating the layout aloud — sign(1) : exponent(8) : mantissa(23) — since FPU and datapath interviews often start here before asking you to design a comparator or

normaliser.

Q19. What is sizeof each expression? (sizeof semantics)

```
int a[10];
int *p = a;
struct { char c; int i; } s;
char str[] = "hardware";

sizeof(a)      sizeof(p)      sizeof(a)/sizeof(a[0])
sizeof(s)      sizeof(str)    sizeof("hardware")
sizeof(a + 0)  sizeof(&a)     sizeof(*&a)
```

Solution

```
sizeof(a)      = 40      (whole array)
sizeof(p)      = 8       (pointer, 64-bit)
sizeof(a)/sizeof(a[0]) = 10 (element count idiom)
sizeof(s)      = 8       (1 + 3 pad + 4)
sizeof(str)    = 9       ("hardware" is 8 chars + '\0')
sizeof("hardware") = 9    (same, the literal is a char[9])
sizeof(a + 0)  = 8       (a DECAYS to int* in the expression)
sizeof(&a)     = 8       (pointer to array is still a pointer)
sizeof(*&a)    = 40      (deref gives the array back)
```

The pair `sizeof(a) = 40` versus `sizeof(a+0) = 8` is the cleanest demonstration that an array is not a pointer but decays to one as soon as it takes part in an expression. Note also `strlen(str) = 8` while `sizeof(str) = 9` — the terminator is storage, not content.

Section 3 — Pointers, volatile & Memory-Mapped Registers

Q20. A control register at 0x40010000 is WRITE-ONLY (reads return garbage). You must set bit 5 without disturbing the other bits. How?

Solution

```
#define CTRL (*(volatile unsigned int *)0x40010000u)

static unsigned int ctrl_shadow = 0;      /* software copy of the reg */

void ctrl_set_bit(int n)
{
    ctrl_shadow |= (1u << n);
    CTRL = ctrl_shadow;                  /* write the whole word */
}
```

Read-modify-write is impossible on a write-only register, so you keep a **shadow variable** in RAM that mirrors what you last wrote, modify that, and write the full word back. Points to make: the shadow must be initialised to the register's reset value, and if an ISR also touches CTRL the shadow update plus write must be a critical section. Write-only registers are common for things like DAC outputs and some clock-gating controls.

Q21. STATUS at 0x40010004 is CLEAR-ON-READ. Explain why this code is broken and fix it.

```
#define STATUS (*(volatile unsigned int *)0x40010004u)
void handle(void)
{
    if (STATUS & ERR_BIT) handle_error();
    if (STATUS & DONE_BIT) handle_done();
}
```

Solution

```
void handle(void)
{
    unsigned int s = STATUS;      /* ONE read, snapshot into a local */
    if (s & ERR_BIT) handle_error();
    if (s & DONE_BIT) handle_done();
}
```

```
}
```

Because STATUS is volatile, each mention is a genuine bus read — and on a clear-on-read register the first read WIPES the flags, so DONE_BIT is lost forever. Read once into a non-volatile local and test the snapshot. This is the flip side of the volatile lesson: volatile guarantees every access happens, so you must control exactly how many accesses you write. Same reasoning applies to FIFO data registers, where each read pops an entry.

Q22. Explain the difference between these three declarations for a peripheral pointer.

```
volatile unsigned int *p1;          /* A */
unsigned int * volatile p2;         /* B */
volatile unsigned int * volatile p3; /* C */
```

Solution

A: pointer to volatile data — the POINTED-TO register may change on its own; the pointer variable itself is ordinary. This is what you want 99% of the time. B: a volatile POINTER to ordinary data — the pointer variable itself might be changed by an ISR or DMA, but the data it points to is normal memory. C: both. Read right-to-left from the name. Getting A vs B backwards means the compiler caches your register reads while pointlessly reloading the pointer — the classic 'I added volatile and it still hangs' bug.

Q23. Write a driver function that works on ANY instance of a UART peripheral, where UART0 is at 0x40011000 and each subsequent instance is 0x1000 higher.

Solution

```
typedef struct {
    volatile unsigned int CTRL;
    volatile unsigned int STATUS;
    volatile unsigned int DATA;
} uart_t;

#define UART(n) ((uart_t *) (0x40011000u + (n) * 0x1000u))

void uart_putc(uart_t *u, char ch)
{
    while ((u->STATUS & TX_READY) == 0) /* poll: volatile re-reads */
        ;
    u->DATA = (unsigned int)ch;
}
/* usage */ uart_putc(UART(0), 'A');  uart_putc(UART(2), 'B');
```

Pass the instance as a `uart_t *` rather than hard-coding addresses — the same code then serves every instance, which is exactly how vendor HAL drivers are written. Note the struct members carry volatile, so the pointer parameter does not need it. Follow-up they may ask: why `while(...)` ; and not a delay? Because polling the actual flag adapts to any baud rate.

Q24. You write to a peripheral configuration register and then immediately to a second peripheral. Why might the writes reach the bus out of order, and what does volatile guarantee?

Solution

`volatile` guarantees the COMPILER will not reorder, merge or eliminate volatile accesses relative to one another, and that each appears exactly once. It guarantees nothing about the hardware below: a write buffer, a bus bridge, or a CPU with out-of-order memory access may still let the second write land first, and on a weakly-ordered core (ARM) the store may still be sitting in a buffer when the next instruction runs.

```
CFG_A = 1;
__asm volatile ("dsb" ::: "memory"); /* data synchronisation barrier */
CFG_B = 1;

/* Common defensive idiom: read back to force completion */
CFG_A = 1;
```



```
(void)CFG_A;      /* read-back flushes the write buffer (if readable) */
```

The read-back trick is used constantly in real drivers, especially before enabling interrupts or after clearing an interrupt flag — otherwise you can return from the ISR before the clear reaches the peripheral and immediately re-enter it.

Q25. TRICKY — What is wrong with accessing a 32-bit register through a 16-bit pointer? Show what happens.

```
volatile unsigned short *p = (volatile unsigned short *)0x40010000u;
unsigned short lo = p[0];
unsigned short hi = p[1];
unsigned int value = ((unsigned int)hi << 16) | lo;
```

Solution

Three problems. (1) Many peripherals only decode 32-bit accesses; a 16-bit read may return garbage, fault, or read a whole word and give you the wrong half. (2) The reassembly assumes little-endian byte order — on big-endian, p[0] holds the HIGH half. (3) Atomicity: the register may change between the two reads, giving a torn value that never actually existed (a real hazard for free-running counters, where the standard fix is read-high, read-low, read-high-again and retry if it changed). Correct approach: access a register at its documented width, once.

Q26. Write a poll-with-timeout helper correctly (Volume 1 showed the broken version). Include the volatile and precedence details.

Solution

```
#define STATUS (*(volatile unsigned int *)0x40010004u)
#define TIMEOUT 100000u /* NO trailing semicolon! */

int wait_ready(void)
{
    for (unsigned i = 0; i < TIMEOUT; i++) {
        if ((STATUS & 0x1u) == 0x1u) /* parens: & is lower than == */
            return 1;                /* success, exit early */
    }
    return 0;                        /* timed out */
}
```

The four things being graded: volatile on the register, the test INSIDE the loop with an early return, parentheses around the mask before comparing, and no stray semicolon after for (. . .) or inside the #define. A production version would also use a hardware timer rather than an iteration count, since a loop count means different wall-clock time at different clock speeds and optimisation levels.

Q27. What does this print? (Pointer arithmetic on a register block)

```
#include <stdio.h>
typedef struct { unsigned int A, B, C, D; } blk_t;
int main(void)
{
    blk_t *b = (blk_t *)0x40000000u;
    unsigned int *w = (unsigned int *)0x40000000u;
    printf("%p\n", (void *) (b + 2));
    printf("%p\n", (void *) (w + 2));
    printf("%p\n", (void *) &b[1].C);
    return 0;
}
```

Solution

Line 1: **0x40000020** — b+2 advances two whole 16-byte structs. Line 2: **0x40000008** — w+2 advances two 4-byte words. Line 3: **0x4000001c** — b[1] starts at +16, and C is the third member at +8 within it, giving 0x40000000 + 16 + 8. Same numeric address, three different step sizes, because pointer arithmetic always scales by the pointed-to type. Mixing a struct pointer and a word pointer over the same register block is normal in drivers, which is exactly why this trips people up.

Section 4 — Output-Tracing Traps

Q28. What is the output? (Short-circuit evaluation with side effects)

```
#include <stdio.h>
int main(void)
{
    int a = 0, b = 5, c = 0;
    if (a++ && b++) c = 1;
    printf("%d %d %d\n", a, b, c);

    int x = 1, y = 5;
    if (x++ || y++) c = 2;
    printf("%d %d %d\n", x, y, c);
    return 0;
}
```

Solution

Line 1: **1 5 0** — `a++` yields 0 (false), so `&&` short-circuits and `b++` NEVER EXECUTES; `b` stays 5. Line 2: **2 5 2** — `x++` yields 1 (true), so `||` short-circuits and `y++` never runs. The right-hand side of `&&` and `||` is not evaluated when the result is already known. This matters in hardware code: `if (ptr && ptr->flag)` is the safe null-check idiom precisely because of short-circuiting, but `if (read_reg() && other())` may skip a register read you assumed happened.

Q29. What is the output? (Signed division and modulo with negatives)

```
#include <stdio.h>
int main(void)
{
    printf("%d %d\n", -7 / 2, -7 % 2);
    printf("%d %d\n", 7 / -2, 7 % -2);
    printf("%d\n", -7 >> 1);
    return 0;
}
```

Solution

Output: **-3 -1**, then **-3 1**, then **-4**. Since C99, integer division truncates toward ZERO, and the remainder takes the sign of the dividend, satisfying $(a/b) * b + a \% b == a$. But arithmetic right shift floors toward negative infinity, so `-7 >> 1` is -4 while `-7/2` is -3. Conclusion for hardware modelling: shifting is NOT the same as dividing for signed values — a genuine source of mismatches between a C golden model and RTL.

Q30. What is the output? (Unsigned loop counter counting down)

```
#include <stdio.h>
int main(void)
{
    int n = 3;
    for (unsigned i = n - 1; i >= 0; i--)
        printf("%u ", i);
    printf("done\n");
    return 0;
}
```

Solution

This loop **never terminates** (it prints `2 1 0` then wraps to 4294967295 and keeps going). An unsigned value is always `>= 0`, so the condition can never be false. Fixes: use a signed counter, or write `for (unsigned i = n; i-- > 0; )`, which tests before decrementing. The mirror-image bug of Volume 1's unsigned `char n < 256` — both come from mismatching counter width/signedness with the comparison range.

Q31. What is the output? (Partial and designated initialisers)

```
#include <stdio.h>
int main(void)
{
```

```

int a[5] = {1, 2};
int b[5] = {0};
int c[5] = {[3] = 7};
struct { int x, y, z; } s = {.y = 4};
printf("%d %d %d %d\n", a[4], b[2], c[3], c[0]);
printf("%d %d\n", s.x, s.y);
return 0;
}

```

Solution

Output: **0 0 7 0** then **0 4**. Rule: if ANY initialiser is present, all unmentioned members are zero-initialised (as if by static initialisation), which is why `= {0}` is the idiomatic way to clear an array or struct. Designated initialisers (C99) set specific elements and zero the rest — very handy for sparse lookup tables and for register-default structs where you only want to name a couple of fields.

Q32. What is the output? (Comparing strings and pointers)

```

#include <stdio.h>
#include <string.h>
int main(void)
{
    char a[] = "abc";
    char b[] = "abc";
    char *p = "abc";
    char *q = "abc";
    printf("%d %d %d\n", a == b, p == q, strcmp(a, b) == 0);
    return 0;
}

```

Solution

Output: **0**, then **usually 1**, then **1**. `a` and `b` are two distinct arrays, so their addresses differ. `p` and `q` point at string literals with identical content, which most compilers merge into one copy in `.rodata` — so the pointers usually compare equal, but this is NOT guaranteed. Never compare strings with `==`; that compares addresses. Use `strcmp` (or `memcmp` for fixed-length fields).

Q33. What is the output? (Order of evaluation of function arguments)

```

#include <stdio.h>
int n = 0;
int next(void) { return ++n; }
int main(void)
{
    printf("%d %d %d\n", next(), next(), next());
    return 0;
}

```

Solution

The output is **unspecified** — commonly **3 2 1** on gcc/x86 (arguments evaluated right to left) but **1 2 3** elsewhere. The C standard does not fix the evaluation ORDER of function arguments, only that each is fully evaluated before the call. The correct interview answer is to say it is unspecified and that you would sequence the calls into separate statements. Same family as Volume 1's `i++ + i++`, but note the difference: this one is unspecified (one of several orders), not undefined (anything at all).

Q34. What is the output? (Static variable inside a recursive function)

```

#include <stdio.h>
int depth(int n)
{
    static int calls = 0;
    calls++;
    if (n <= 1) return calls;
    return depth(n - 1);
}
int main(void)
{

```

```

    printf("%d ", depth(3));
    printf("%d\n", depth(3));
    return 0;
}

```

Solution

Output: **3 6**. The static counter is shared by ALL invocations, including recursive ones, and persists between the two top-level calls: the first call adds 3, the second adds 3 more. Static locals in recursive functions are a classic source of 'works the first time only' bugs — the same hazard as a driver function that keeps state in a static instead of a per-instance struct.

Q35. What is the output? (Ternary operator type promotion)

```

#include <stdio.h>
int main(void)
{
    int i = 5;
    float f = 2.0f;
    printf("%d\n", (int)(1 ? i : f));
    printf("%zu\n", sizeof(1 ? i : f));
    printf("%zu\n", sizeof(1 ? 'a' : 0));
    return 0;
}

```

Solution

Output: **5**, then **4**, then **4**. The ternary has ONE result type, determined by the usual arithmetic conversions across BOTH branches, regardless of which branch is taken: int and float give float (4 bytes), so line 2 is sizeof(float). Line 3: char and int give int, so 4 in C (this is 4 in C++ too here, since 0 is an int). The value 5 is converted to 5.0f then back to int. Relevant when a ternary picks between differently-typed register values — the narrower one is silently promoted.

Section 5 — Hardware-Flavored Coding Problems

Q36. Write a rising-edge and falling-edge detector for a sampled input signal. One call = one clock.

Solution

```

typedef struct { int prev; } edge_t;

int rising(edge_t *e, int cur)
{
    int r = (cur && !e->prev);    /* 0 -> 1 transition */
    e->prev = cur;
    return r;
}

int falling(edge_t *e, int cur)
{
    int f = (!cur && e->prev);    /* 1 -> 0 transition */
    e->prev = cur;
    return f;
}

```

Exactly the RTL idiom `assign rise = cur & ~prev;` with prev being a registered copy of cur. Keeping prev in a struct rather than a static lets you instantiate one detector per signal — the C equivalent of instantiating the module twice. Say that when they ask 'what if I have 8 inputs?'.

Q37. Write a switch debouncer that reports a stable level only after N consecutive identical samples.

Solution

```

#define STABLE_N 4

```

```
typedef struct { unsigned char hist; int level; } debounce_t;

int debounce(debounce_t *d, int raw)    /* call at a fixed sample rate */
{
    d->hist = (unsigned char)((d->hist << 1) | (raw & 1));
    unsigned char m = (unsigned char)((1u << STABLE_N) - 1u); /* 0x0F */
    if ((d->hist & m) == m) d->level = 1; /* N ones in a row */
    else if ((d->hist & m) == 0) d->level = 0; /* N zeros in a row */
    return d->level; /* otherwise hold */
}
```

A shift register of the last 8 samples plus a mask test for 'all ones' or 'all zeros' — no counters needed. This is precisely how a hardware debouncer is built, and it reuses the shift-register idiom from Volume 1's sequence detector. Mention the sample period (typically 1–10 ms) since debounce time = $N \times$ sample period.

Q38. Write a clock divider: given a tick function called every input clock, produce an output that toggles every N ticks (divide by $2N$), and one that pulses for a single tick every N .

Solution

```
typedef struct { unsigned cnt; int out; } div_t_;

int div_toggle(div_t_ *d, unsigned n)    /* 50% duty, period 2N */
{
    if (++d->cnt >= n) { d->cnt = 0; d->out = !d->out; }
    return d->out;
}

int div_pulse(div_t_ *d, unsigned n)    /* 1-tick pulse every N */
{
    if (++d->cnt >= n) { d->cnt = 0; return 1; }
    return 0;
}
```

Two different meanings of 'divide by N ' that interviewers deliberately blur: a toggling divider gives a 50% duty cycle but halves the frequency again (period $2N$), while a pulse/strobe generator gives one narrow enable every N cycles. In RTL the second is the common 'clock enable' pattern — you rarely generate a real divided clock in an ASIC because it creates a new clock domain.

Q39. Implement a Gray-code counter: maintain a binary count internally and output the Gray code, then verify successive outputs differ in exactly one bit.

Solution

```
typedef struct { unsigned bin; } gray_ctr_t;

unsigned gray_next(gray_ctr_t *g, unsigned width)
{
    g->bin = (g->bin + 1u) & ((1u << width) - 1u); /* wrap at width */
    return g->bin ^ (g->bin >> 1); /* bin -> gray */
}

/* self-check, as you would assert in a testbench */
int one_bit_change(unsigned a, unsigned b)
{
    unsigned d = a ^ b;
    return d && !(d & (d - 1));
}
```

The standard construction: count in binary (easy to increment) and convert to Gray on the output (one XOR). Counting directly in Gray is painful, which is why real async FIFOs do exactly this. The self-check ties back to Section 1 Q8 — and a wrap from the maximum Gray value back to 0 is also a single-bit change, which is what makes Gray-coded FIFO pointers safe across clock domains.

Q40. Implement a round-robin arbiter over an 8-bit request vector: grant one request, rotating priority so the last-granted requester has the lowest priority next time.

Solution

```
unsigned rr_grant(unsigned req, unsigned *last) /* returns one-hot grant */
{
    if (req == 0) return 0;
    for (unsigned i = 1; i <= 8; i++) {
        unsigned idx = (*last + i) & 7u; /* start just after last */
        if (req & (1u << idx)) {
            *last = idx;
            return 1u << idx;
        }
    }
    return 0;
}
```

Start scanning one position past the previous winner and wrap — that single rule is what makes it fair, guaranteeing no requester starves. Contrast with a fixed-priority arbiter, which is just `req & -req` (lowest set bit) and can starve high-index requesters. Expect the follow-up 'how would you build this combinationaly?' — answer: rotate the request vector by `last+1`, take the lowest set bit, rotate the grant back.

Q41. Implement a bit-serial CRC-8 (polynomial 0x07, MSB-first) that processes one byte at a time.

Solution

```
unsigned char crc8_byte(unsigned char crc, unsigned char data)
{
    crc ^= data; /* XOR the byte into the register */
    for (int i = 0; i < 8; i++) {
        if (crc & 0x80u) /* MSB set -> shift and XOR poly */
            crc = (unsigned char)((crc << 1) ^ 0x07u);
        else
            crc = (unsigned char)(crc << 1);
    }
    return crc;
}

unsigned char crc8(const unsigned char *p, int n)
{
    unsigned char c = 0x00; /* or 0xFF, per the spec */
    while (n--) c = crc8_byte(c, *p++);
    return c;
}
```

This IS an LFSR: shift left, and when the bit shifted out is 1, XOR the polynomial taps back in — division in GF(2). The hardware version does all 8 iterations combinationaly in one clock. Say that CRC catches burst errors that a simple additive checksum misses, and that a byte-at-a-time table lookup is the speed optimisation if asked.

Q42. Implement an 8-tap moving-average filter using a circular buffer, with no division instruction.

Solution

```
#define TAPS 8 /* power of two! */

typedef struct { int buf[TAPS]; int idx; int sum; } avg_t;

int avg_push(avg_t *a, int sample)
{
    a->sum -= a->buf[a->idx]; /* remove the oldest */
    a->buf[a->idx] = sample; /* store the newest */
    a->sum += sample;
    a->idx = (a->idx + 1) & (TAPS - 1);
    return a->sum >> 3; /* divide by 8 = shift */
}
```

```
}
```

Three hardware-friendly choices: a running sum so each update is $O(1)$ rather than re-summing 8 taps; a power-of-two tap count so the divide becomes a shift; and a mask instead of % for the circular index. Caveat worth raising: $\gg 3$ floors toward negative infinity for signed samples, so add a rounding term $((\text{sum} + 4) \gg 3)$ if unbiased rounding matters.

Q43. Implement a software PWM generator: given a duty value 0..255, produce the output level for each tick of a free-running 8-bit counter.

Solution

```
typedef struct { unsigned char cnt; } pwm_t;

int pwm_tick(pwm_t *p, unsigned char duty)
{
    int out = (p->cnt < duty);    /* compare counter against duty */
    p->cnt++;                      /* wraps 255 -> 0 automatically */
    return out;
}
```

The whole of PWM is one comparator against a free-running counter — duty 0 gives always-low, duty 255 gives high for 255 of 256 ticks. Note the 8-bit counter wraps by itself, which is why the type choice matters (an unsigned int counter would need an explicit mask). Interviewers like this because the C is trivial but it checks whether you understand the underlying digital block.

Section 6 — Structs, Unions & Register Maps

Q44. Define a union that lets you access a control register either as one 32-bit word or as named bit-fields. What is the portability caveat?

Solution

```
typedef union {
    unsigned int word;
    struct {
        unsigned int enable : 1;    /* bit 0 */
        unsigned int mode   : 2;    /* bits [2:1] */
        unsigned int        : 5;    /* reserved */
        unsigned int divisor : 8;    /* bits [15:8] */
        unsigned int        : 16;
    } bits;
} ctrl_u;

ctrl_u c;
c.word = 0; c.bits.mode = 3; c.bits.divisor = 12;
REG = c.word;    /* single 32-bit write */
```

Convenient and readable, and the single write at the end avoids multiple partial register writes. Caveats: bit-field allocation order within a unit is implementation-defined (LSB-first on gcc/ARM, but not guaranteed), and in C++ reading a different union member than you wrote is formally UB. That is why safety-critical and multi-toolchain codebases prefer explicit mask/shift macros — this union is fine when you control the compiler.

Q45. Why is memcmp the wrong way to compare two structs? Show a case where two logically equal structs compare unequal.

```
struct pkt { char type; int len; };
struct pkt a, b;
a.type = 1; a.len = 100;
b.type = 1; b.len = 100;
if (memcmp(&a, &b, sizeof a) == 0) puts("equal");
```

Solution

The 3 padding bytes between `type` and `len` hold INDETERMINATE values — whatever was on the stack — and `memcmp` compares them too, so two logically identical structs can compare unequal. (Conversely, `memcmp` can report equality for structs holding different pointers to identical data.) Correct approach: compare member by member. If you truly need byte comparison — for hashing or a golden-model checksum — zero the struct first with `memset(&a, 0, sizeof a)` or use a designated initialiser, which zero-fills padding on most implementations but is still not guaranteed by the standard.

Q46. Design a struct for a variable-length packet: a fixed 4-byte header followed by *n* data bytes, allocated as ONE block.

Solution

```
typedef struct {
    unsigned short id;
    unsigned short len;          /* number of bytes in data[] */
    unsigned char data[];       /* flexible array member (C99) */
} packet_t;

packet_t *make_packet(unsigned short id, const unsigned char *src, unsigned short n)
{
    packet_t *p = malloc(sizeof(packet_t) + n); /* header + payload */
    if (!p) return NULL;
    p->id = id; p->len = n;
    memcpy(p->data, src, n);
    return p;                                /* single free() later */
}
```

The flexible array member must be LAST and does not count toward `sizeof` (here `sizeof(packet_t)` is 4). One allocation means one free, contiguous memory suitable for DMA, and no pointer chasing. This is how network and bus-transaction descriptors are modelled in C. Before C99 people wrote `data[1]` and over-allocated — the 'struct hack'; mention it if they ask about older code.

Q47. Why must a register-map struct use fixed-width types from `stdint.h` rather than `int/long`? Give a concrete failure.

Solution

```
/* WRONG - sizes vary by platform */
typedef struct { unsigned long CTRL; unsigned int STATUS; } bad_t;

/* RIGHT - exact widths, guaranteed offsets */
#include <stdint.h>
typedef struct { volatile uint32_t CTRL; volatile uint32_t STATUS; } good_t;
```

`unsigned long` is 4 bytes on 32-bit Linux and Windows-64, but 8 bytes on 64-bit Linux. Build the same driver for a host-based simulator on 64-bit Linux and `STATUS` silently moves from offset 4 to offset 8 — every register access after the first is wrong, and it compiles cleanly. Fixed-width types make the layout match the hardware datasheet on every toolchain. Same reason DPI/SystemC interfaces to a testbench must use `uint32_t`.

Q48. What is the output? (Nested structs and offset arithmetic)

```
#include <stdio.h>
#include <stddef.h>
typedef struct { unsigned int lo, hi; } pair_t;
typedef struct { unsigned char tag; pair_t p; unsigned int crc; } frame_t;
int main(void)
{
    printf("%zu %zu %zu %zu\n",
           sizeof(pair_t), sizeof(frame_t),
           offsetof(frame_t, p), offsetof(frame_t, crc));
    return 0;
}
```

Solution

Output: **8 16 4 12**. `pair_t` is two 4-byte words with no padding. In `frame_t`, `tag` takes 1 byte then 3 padding bytes so the nested struct starts at offset 4 (it inherits `pair_t`'s 4-byte alignment); `p` occupies 4..11; `crc` sits at 12; total 16 with no trailing padding needed. Nested structs propagate their alignment requirement upward — a detail that catches people when a packet header suddenly gains hidden bytes.

Q49. What does `__attribute__((packed))` do, and what new hazard does it introduce?

```
struct __attribute__((packed)) hdr {
    unsigned char  type;      /* offset 0 */
    unsigned int   addr;      /* offset 1 - UNALIGNED! */
    unsigned short len;       /* offset 5 */
};                             /* sizeof == 7, not 12 */
```

Solution

Packing removes all padding so the struct matches a wire/register format byte for byte — essential for protocol headers. The hazard: members are no longer naturally aligned, so `&h.addr` is a misaligned pointer. On ARMv7-M or MIPS, dereferencing it can fault or silently return rotated data; the compiler usually generates safe byte-by-byte accesses for direct member access, but taking the ADDRESS and passing it elsewhere loses that protection. Safer alternative: keep the struct unpacked and serialise with explicit shifts, which is also endianness-safe.

Q50. Model a peripheral with 8 identical channels, each having 4 registers, and write a function to configure channel `n`.

Solution

```
typedef struct {
    volatile uint32_t CTRL;
    volatile uint32_t ADDR;
    volatile uint32_t LEN;
    volatile uint32_t STATUS;
} chan_t;                               /* 16 bytes per channel */

typedef struct {
    volatile uint32_t GLOBAL_CTRL;
    volatile uint32_t RESERVED[3];      /* keep offsets exact */
    chan_t            chan[8];          /* array of channels */
} dma_t;

#define DMA ((dma_t *)0x40026000u)

void dma_setup(int n, uint32_t src, uint32_t len)
{
    DMA->chan[n].ADDR = src;
    DMA->chan[n].LEN  = len;
    DMA->chan[n].CTRL = 1u;              /* start last! */
}
```

An array of structs mirrors a repeated hardware block exactly, and `chan[n]` generates `base + n*16` for free. Two details interviewers check: the `RESERVED` array preserving the offset of the channel array, and writing the enable/start bit LAST so the channel does not begin with half-configured parameters — an ordering requirement that also justifies a barrier on weakly-ordered cores.

Q51. What is the output? (Struct passed by value vs array inside a struct)

```
#include <stdio.h>
typedef struct { int v[3]; } vec_t;
vec_t bump(vec_t a) { a.v[0] += 100; return a; }
int main(void)
{
    vec_t x = {1, 2, 3};
    vec_t y = bump(x);
    printf("%d %d\n", x.v[0], y.v[0]);
    int raw[3] = {1,2,3};
    /* int raw2[3] = raw; */           /* would not compile */
}
```

```

    printf("%zu\n", sizeof(vec_t));
    return 0;
}

```

Solution

Output: **1 101** then **12**. Wrapping an array in a struct gives it value semantics: bump receives a full COPY (arrays inside structs are copied, unlike bare arrays which decay to pointers), so x is unchanged, and structs can be assigned and returned whereas raw arrays cannot. This wrapper trick is how you pass fixed-size buses/vectors around by value in C models — at the cost of copying, so use pointers for large structs.

Section 8 — More Bit Tricks & Digital-Design Idioms

Q52. Compute the average of two unsigned integers WITHOUT overflowing, i.e. avoid (a+b)/2 wrapping.

Solution

```

unsigned avg(unsigned a, unsigned b)
{
    return (a & b) + ((a ^ b) >> 1);    /* floor((a+b)/2), no overflow */
}
/* alternative: */ return a + ((b - a) >> 1); /* requires b >= a */

```

Decompose the addition: $a \& b$ is the carry contribution (bits set in both, each worth 2), and $a \wedge b$ is the sum without carries; shifting the latter halves it. No intermediate exceeds 32 bits. This exact bug — $(lo+hi)/2$ overflowing in binary search — famously sat in the JDK for years, and in hardware it is the reason midpoint calculations use a carry-save form.

Q53. Detect whether a 32-bit word contains a zero BYTE, in constant time (the trick behind word-at-a-time strlen).

Solution

```

#define HASZERO(v) (((v) - 0x01010101u) & ~(v) & 0x80808080u)

int has_zero_byte(unsigned v) { return HASZERO(v) != 0; }

```

Subtracting 1 from each byte lane borrows into bit 7 of that lane only when the byte was 0x00; ANDing with $\sim v$ ensures the byte was not already 0x80-something, and the 0x80808080 mask keeps only the lane flags. Result nonzero means at least one byte was zero. This is how optimised strlen/memchr scan 4 or 8 bytes per iteration — the software version of a per-lane comparator with a wide OR reduction.

Q54. Reverse the bits WITHIN each byte of a 32-bit word, leaving the byte order unchanged.

Solution

```

unsigned reverse_bits_in_bytes(unsigned x)
{
    x = ((x & 0x55555555u) << 1) | ((x >> 1) & 0x55555555u); /* pairs */
    x = ((x & 0x33333333u) << 2) | ((x >> 2) & 0x33333333u); /* nibbles */
    x = ((x & 0x0F0F0F0Fu) << 4) | ((x >> 4) & 0x0F0F0F0Fu); /* bytes */
    return x;
}

```

This is Volume 1's full 32-bit reversal with the last two stages (byte swap and half-word swap) OMITTED — stopping early reverses within each byte only. Useful for protocols that transmit LSB-first within a byte but send bytes in order, such as SPI/UART framing, JPEG bit order, and some serialiser conventions. Being able to explain why dropping the final stages changes the granularity shows real understanding of the divide-and-conquer structure.

Q55. Interleave the bits of two 8-bit values a and b to form a 16-bit Morton number (b7 a7 b6 a6 ... b0 a0).

Solution

```
static unsigned spread(unsigned x)      /* 0000abcd -> 0a0b0c0d */
{
    x = (x | (x << 4)) & 0xF0Fu;
    x = (x | (x << 2)) & 0x3333u;
    x = (x | (x << 1)) & 0x5555u;
    return x;
}

unsigned morton(unsigned char a, unsigned char b)
{
    return spread(a) | (spread(b) << 1);
}
```

The spread function pushes each bit into every other position using the same halving cascade as bit reversal. Morton (Z-order) interleaving is used in memory address scrambling, texture/cache tiling and multi-bank DRAM address mapping — so it shows up in memory-controller interviews. Explain the cascade rather than memorising the constants: each step doubles the gap between bits.

Q56. Sign-extend a value of ARBITRARY width w (generalise Volume 1's 12-bit case), with a branchless form.

Solution

```
int sext(unsigned v, int w)            /* v holds w valid bits, w in 1..32 */
{
    unsigned m = 1u << (w - 1);        /* the sign bit of the field */
    v &= (w == 32) ? 0xFFFFFFFFu : ((1u << w) - 1u);
    return (int)((v ^ m) - m);          /* branchless sign extension */
}
```

The identity $(v \wedge m) - m$: if the sign bit is clear, XOR sets it and the subtraction removes it, leaving v ; if set, XOR clears it and subtracting m pushes the value negative by exactly 2^w . No shifts, so no reliance on implementation-defined arithmetic right shift. Note the $w == 32$ guard — the same shift-by-width UB from Section 1 Q1.

Q57. Count the number of TRAILING ones in a word, and the number of leading ones.

Solution

```
int trailing_ones(unsigned x)
{
    x = ~x;                             /* trailing ones become trailing zeros */
    if (x == 0) return 32;
    int n = 0;
    while ((x & 1u) == 0) { n++; x >>= 1; }
    return n;
}

int leading_ones(unsigned x) { return leading_zeros(~x); } /* clz of ~x */
```

The whole trick is complementing: counting ones becomes counting zeros, for which you already have CTZ/CLZ. Interviewers use this to see whether you reduce a new problem to a solved one instead of writing fresh logic. Practical use: finding the length of a run of set bits in an allocation bitmap or a bus grant vector.

Q58. Compute the absolute difference $|a - b|$ of two unsigned values without branching, and explain the overflow concern.

Solution

```
unsigned absdiff(unsigned a, unsigned b)
{
    unsigned d = a - b;                  /* wraps if a < b */
    unsigned m = (unsigned)-(a < b);    /* all ones when a < b */
    return (d ^ m) - m;                  /* conditional negate */
}
```

Unsigned subtraction wraps modulo 2^{32} rather than being undefined, so $a - b$ is safe even when negative in the mathematical sense — the wrapped value is exactly the two's-complement of the true difference, and the conditional-negate identity fixes it. For SIGNED inputs you must be careful: $a - b$ itself can overflow (INT_MIN vs INT_MAX), so widen to 64-bit or compare first. Absolute difference is a core operation in SAD units for video hardware.

Q59. Implement a 32-bit population count using the SWAR (parallel bit-counting) method, and explain why it beats the loop.

Solution

```
int popcount_swar(unsigned x)
{
    x = x - ((x >> 1) & 0x55555555u);           /* pairs */
    x = (x & 0x33333333u) + ((x >> 2) & 0x33333333u); /* nibbles */
    x = (x + (x >> 4)) & 0x0F0F0F0Fu;           /* bytes */
    return (int)((x * 0x01010101u) >> 24);       /* sum lanes */
}
```

SWAR = SIMD Within A Register: each step adds counts in parallel lanes, halving the number of lanes and doubling their width — 2-bit counts, then 4-bit, then 8-bit — and the final multiply sums all four byte lanes into the top byte in one operation. Fixed 12 or so operations regardless of the data, versus up to 32 iterations for the loop. This is literally an adder tree, which is why hardware popcount units have log-depth.

Q60. Find the next larger integer with the SAME number of set bits (used for enumerating combinations, e.g. all k-hot masks).

Solution

```
unsigned next_same_popcount(unsigned x)          /* 'snoob' / Gosper's hack */
{
    unsigned c = x & (unsigned)(-(int)x);        /* lowest set bit */
    unsigned r = x + c;                          /* ripple the carry */
    return r | (((x ^ r) >> 2) / c);              /* re-place the low ones */
}
/* 0b00111 -> 0b01011 -> 0b01101 -> 0b01110 -> 0b10011 ... */
```

Add the lowest set bit to ripple the rightmost block of ones up one position, then re-insert the leftover ones at the bottom, right-justified — the division by c performs that right-justification. Use case in verification: enumerating every k-hot test vector for an arbiter or decoder without generating all 2^n combinations. Do not memorise it blindly; be able to trace one step on 0b00111.

Section 9 — More Output-Tracing & Memory Traps

Q61. What is the output of each version, and which one is undefined?

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char a[] = "abcdef";
    memmove(a + 1, a, 4);
    printf("%s\n", a);

    char b[] = "abcdef";
    memcpy(b + 1, b, 4);          /* overlapping! */
    printf("%s\n", b);
    return 0;
}
```

Solution

memmove prints **aabcdf**: it handles overlap correctly by copying backwards when the destination is above the source, so 'abcd' lands at positions 1..4. The memcpy version is **undefined behaviour** — it may print the same thing, or produce 'aaaaaf' if the implementation copies forwards byte by byte, or something else

entirely with vectorised copies. Rule: any time source and destination might overlap (shifting a buffer, sliding a window), use `memmove`.

Q62. What is wrong with this function? What does the caller see?

```
#include <stdio.h>
char *fmt_id(int id)
{
    static char buf[16];
    snprintf(buf, sizeof buf, "ID-%d", id);
    return buf;
}
int main(void)
{
    printf("%s %s\n", fmt_id(1), fmt_id(2));
    return 0;
}
```

Solution

Both pointers refer to the SAME static buffer, so the second call overwrites the first result before `printf` runs — the likely output is **ID-2 ID-2** (and the order of the two calls is unspecified, so it could be ID-1 ID-1). The static buffer at least avoids the dangling-pointer bug of returning a local array, but it is not reentrant: two threads, or an ISR, would corrupt each other. Correct API design: have the caller pass the buffer and its size, which is why the standard library is full of functions shaped like `snprintf(buf, n, ...)`.

Q63. What is the bug? (realloc semantics)

```
int *grow(int *p, int old_n, int new_n)
{
    int *q = realloc(p, new_n * sizeof(int));
    for (int i = old_n; i < new_n; i++)
        p[i] = 0;          /* <-- ? */
    return q;
}
```

Solution

After a successful `realloc` the old pointer `p` is INVALID — `realloc` may have moved the block, so writing through `p` is a use-after-free. Write through `q` instead. Second bug: `realloc` returns NULL on failure while leaving the original block allocated, so `p = realloc(p, n)` leaks the original if it fails; always assign to a temporary and check it before overwriting `p`. Third, subtler: `new_n * sizeof(int)` can overflow for large `n` — check the count first.

Q64. What is the output? (sizeof on a pointer parameter — a real security bug class)

```
#include <stdio.h>
#include <string.h>
void clear(char *buf) { memset(buf, 0, sizeof(buf)); }
int main(void)
{
    char data[32];
    memset(data, 'A', sizeof data);
    clear(data);
    printf("%d %d\n", data[0], data[31]);
    return 0;
}
```

Solution

Output: **0 65**. Inside `clear`, `buf` is a POINTER, so `sizeof(buf)` is 8 (64-bit) and only the first 8 bytes are cleared — `data[31]` is still 'A' (65). This is a well-known CWE (incorrect `sizeof` on a pointer) and appears in real CVEs where sensitive data was not actually wiped. Fix: pass the length explicitly, `void clear(char *buf, size_t n)`. Compile with `-Wsizeof-pointer-memaccess` and gcc will warn.

Q65. What is the output? (Out-of-bounds access reading adjacent storage)

```
#include <stdio.h>
int main(void)
{
    int a[3] = {1, 2, 3};
    int b[3] = {7, 8, 9};
    printf("%d\n", a[3]);          /* one past the end */
    printf("%d\n", *(a + 3));
    return 0;
}
```

Solution

Both lines are **undefined behaviour**. In practice they often print 7 (or some stack value) because `b` happens to sit right after `a`, which is exactly what makes this bug so dangerous: it appears to 'work', passes your tests, and then breaks when the compiler reorders locals or you add a variable. Note the subtlety: forming the ADDRESS `a + 3` (one past the end) is legal and is how loop bounds work; DEREFERENCING it is not.

Q66. What is the bug in this list cleanup, and what is the output risk?

```
void free_list(node_t *head)
{
    for (node_t *p = head; p != NULL; p = p->next)
        free(p);
}
```

Solution

```
void free_list(node_t *head)
{
    node_t *p = head;
    while (p) {
        node_t *next = p->next; /* save BEFORE freeing */
        free(p);
        p = next;
    }
}
```

The original reads `p->next` AFTER `p` has been freed — a use-after-free. It frequently 'works' because most allocators do not immediately scrub the block, which is why the bug survives testing and then corrupts the heap under load. Same discipline as reversing a linked list: save the pointer you are about to destroy before you destroy it.

Q67. What is the output? (strncpy does not always terminate)

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char dst[4];
    memset(dst, 'X', sizeof dst);
    strncpy(dst, "abcdef", 4);
    printf("[%4s] %d\n", dst, dst[3]);
    return 0;
}
```

Solution

Prints **[abcd] 100** — `dst` holds 'a','b','c','d' with NO terminating '\0' (`dst[3]` is 'd' = 100). Passing `dst` to `printf("%s")` or `strlen` would then run off the end. `strncpy` copies at most `n` bytes and only NUL-terminates if the source is shorter than `n` (in which case it pads the remainder with zeros — an odd, historical behaviour). Safer: `snprintf(dst, sizeof dst, "%s", src)`, which always terminates.

Q68. What is the output? (Uninitialised vs zero-initialised storage)

```
#include <stdio.h>
int g_arr[4];          /* global */
static int s_val;      /* static */
int main(void)
{
```

```

    int l_arr[4];                /* local, uninitialised */
    printf("%d %d\n", g_arr[2], s_val);
    printf("%d\n", l_arr[2]);
    return 0;
}

```

Solution

Line 1: **0 0** — objects with static storage duration (globals and statics) are guaranteed zero-initialised; the startup code zeroes the .bss section before main. Line 2 is **indeterminate** — a local has whatever was on the stack. Firmware relevance: that .bss zeroing is done by the startup file, so if you write your own reset handler and forget the loop, every global silently starts as garbage — a genuinely painful bring-up bug.

Q69. Spot the FOUR problems in this buffer-copy routine.

```

char *copy_data(const char *src, int n)
{
    char *dst = malloc(n);          /* line 1 */
    strcpy(dst, src);               /* line 2 */
    dst[n] = '\0';                  /* line 3 */
    free(dst);                      /* line 4 */
    return dst;                     /* line 5 */
}

```

Solution

(1) No NULL check on malloc — dereferencing on allocation failure crashes. (2) strcpy copies until the source's terminator regardless of n, so if src is longer than n it overflows the heap block; use memcpy with n, or strncpy/snprintf. (3) dst[n] is one past the end of an n-byte block — you must allocate n+1 to hold a terminator. (4) The buffer is freed and then the dangling pointer is returned, so every caller has a use-after-free. Corrected: allocate n+1, check it, memcpy n bytes, terminate at index n, and do NOT free before returning.

How to use Volume 2: treat it as a mock exam rather than a tutorial. Give yourself 3–4 minutes per question, write the answer out fully (code or a traced value), and only then read the solution. Score yourself: anything you got wrong, go back to the matching section of Volume 1, re-derive the underlying technique, and return to the Volume 2 question a day later. The eight sections here map one-to-one onto Volume 1, so a weak score in a section tells you exactly which page to revisit.

The five techniques that generate most of these answers: (1) mask-and-shift for any field access; (2) $x \& (x-1)$ and $x \& -x$ for lowest-set-bit work; (3) all-ones masks via $\sim (flag)$ for branchless selection; (4) save-before-you-overwrite when manipulating pointers; (5) read a volatile register exactly once into a local before testing it. If those five are reflexes, most unseen questions become mechanical.